

Vera Challenge Bot — Build, Run & Deploy Guide

Companion to `bot.py` / `conversation_handlers.py` / `generate_submission.py` / `README.md`

This walks through the four files already generated for you, end to end: install, run locally, self-test with magicpin's harness, generate `submission.jsonl`, deploy to a public URL, and submit.

1. What you already have

File	What it is
<code>bot.py</code>	The FastAPI server — all 5 endpoints + the composer + conversation state machine
<code>conversation_handlers.py</code>	Optional multi-turn <code>respond()</code> wrapper (deliverable \$7.4)
<code>generate_submission.py</code>	Builds <code>submission.jsonl</code> from <code>dataset/</code> + <code>test_pairs.json</code>
<code>README.md</code>	Approach, tradeoffs, what to hand in

2. One-time setup

Put all four files in one folder, alongside magicpin's own `generate_dataset.py` and `judge_simulator.py` (from your uploads). Then:

```
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
```

```
pip install fastapi uvicorn httpx
```

Optional but recommended — an LLM key gives you the real composition quality (the bot works without one, using a template fallback, but that's for structural testing only):

```
export ANTHROPIC_API_KEY=sk-ant-... # or OPENAI_API_KEY, then set LLM_PROVIDER=openai
```

3. Run the bot locally

```
uvicorn bot:app --host 0.0.0.0 --port 8080
```

Check it's alive:

```
curl http://localhost:8080/v1/healthz
# {"status":"ok","uptime_seconds":2,"contexts_loaded":{"category":0,"merchant":0,"customer":0,"trigger":0}}
```

4. Generate the dataset

magicpin's `generate_dataset.py` expands seed JSON files (provided separately by the challenge — not among the files you uploaded to me) into the full `dataset/` folder plus `test_pairs.json`.

```
python3 generate_dataset.py --seed-dir . --out dataset
```

This creates `dataset/categories`, `dataset/merchants`, `dataset/customers`, `dataset/triggers`, and `dataset/test_pairs.json` (the 30 canonical test pairs).

5. Self-test with `judge_simulator.py`

With the bot still running in one terminal, open `judge_simulator.py` and fill in its CONFIGURATION section (`BOT_URL`, `LLM_PROVIDER`, `LLM_API_KEY` — this is the judge's own LLM key, used to play the merchant and score you, separate from your bot's key):

```
BOT_URL = "http://localhost:8080"
LLM_PROVIDER = "anthropic"      # or openai / gemini / deepseek / groq / ollama
LLM_API_KEY = "sk-..."
TEST_SCENARIO = "all"
export BOT_URL=http://localhost:8080
python judge_simulator.py
```

It prints each scenario's prompts, your bot's responses, and a mock score. Iterate on `bot.py` (prompt wording, `TRIGGER_FRAMING` hints, the regex lists in `handle_reply`) until scores look healthy, then move on.

6. (Optional) Manual endpoint checks with curl

Useful when debugging one endpoint in isolation, without running the full simulator:

```
# Push a context
curl -X POST -H "Content-Type: application/json" \
  -d @dataset/categories/dentists.json \
  http://localhost:8080/v1/context

# Fire a tick
curl -X POST -H "Content-Type: application/json" \
  -d '{"now":"2026-04-26T10:35:00Z","available_triggers":["trg_001..."]}' \
  http://localhost:8080/v1/tick

# Send a merchant reply
curl -X POST -H "Content-Type: application/json" \
  -d '{"conversation_id":"conv_001","merchant_id":"m_001...","from_role":"merchant",
      "message":"Yes please","received_at":"2026-04-26T10:42:00Z","turn_number":2}' \
  http://localhost:8080/v1/reply
```

7. Build submission.jsonl

This calls `compose_message()` directly (no HTTP needed) against the 30 canonical test pairs:

```
python3 generate_submission.py --dataset dataset --out submission.jsonl
```

Sanity-check a few lines before submitting:

```
head -n 3 submission.jsonl | python3 -m json.tool
```

8. Deploy to a public URL

The judge needs a public HTTPS (or HTTP) URL reachable during your test window — any of these work:

- **Render / Railway / Fly.io** — connect the repo, set start command `uvicorn bot:app --host 0.0.0.0 --port $PORT`, add your LLM key as an env var.
- **A cloud VM** (AWS/GCP/Azure) — run `uvicorn` behind a process manager (`systemd/pm2`) or just `screen/tmux` for a short test window.
- **ngrok tunnel** for the fastest path during dev: run the bot locally, then `ngrok http 8080` and use the https URL it prints.
- **Replit** — paste the files in, add a run command, use its always-on URL.

Whichever you pick, re-run the healthz + judge_simulator checks against the public URL before submitting — a bot that only works on localhost will fail warmup.

9. Pre-flight checklist (from the testing brief)

- Endpoint reachable from the public internet
- All 5 endpoints return correct schemas (healthz, metadata, context, tick, reply)
- /v1/context is idempotent on (scope, context_id, version)
- /v1/tick returns within 30s even with nothing to send
- /v1/reply returns within 30s for any conversation
- Bot persists context across calls without restarting
- judge_simulator.py passes locally with non-zero scores
- LLM API quota/rate limits set so the bot survives a 60-min test window
- Submitted the public URL via the submission portal, plus README.md and submission.jsonl

10. Troubleshooting

Symptom	Likely cause / fix
healthz shows 0 contexts after warmup	Judge's context pushes never reached the bot — check the deployed URL, not localhost
/v1/tick always returns empty actions	suppression_key already fired (SUPPRESSED set) — expected on 2nd+ tick for the same context
Bot times out on /v1/tick or /v1/reply	LLM call too slow — check LLM_TIMEOUT_S in bot.py, or fall back sooner
Same body sent twice (anti-repetition penalty)	No LLM key configured — the template fallback doesn't read conversation history; add a key
500 error on a request	Check server logs (uvicorn stdout) — most likely a schema mismatch in the pushed context

11. Submit

Per challenge-brief.md §7, hand in: your public bot URL, submission.jsonl (30 lines), README.md (already written), and optionally conversation_handlers.py. Double-check /v1/metadata reflects your real team name, model, and approach before submitting.